1. What is a Class Method?

A **class method** is a method that:

- Belongs to the **class** (not to a specific object)
- Takes **cls** as its first parameter instead of self
- Can **access or modify class-level variables**

🧱 **Syntax**

```
class ClassName:
    @classmethod
    def method_name(cls, args):
        # code here
```

cls → refers to the **class itself** (like self refers to the object).

⚙️ **2. Example: Understanding Class Method**

Let's start with a simple example 👇

```
class Employee:
    company_name = "TechCorp"   # class variable

    def __init__(self, name, salary):
        self.name = name        # instance variable
        self.salary = salary

    @classmethod
    def change_company(cls, new_name):
        cls.company_name = new_name   # modify class variable
```

```
# Create two employees
e1 = Employee("Alice", 50000)
e2 = Employee("Bob", 60000)

print(Employee.company_name)  # TechCorp

# Change company name using class method
Employee.change_company("NextGenSoft")

print(e1.company_name)  # NextGenSoft
print(e2.company_name)  # NextGenSoft
```

✅ Output:

```
TechCorp
NextGenSoft
NextGenSoft
```

🧠 **Both objects reflect the change**, since it's made at the **class level**.

🧩 3. Why Use Class Methods?

You use **class methods** when:

- You need to **access or modify class variables**
- You want to create **alternative constructors** (more on this next!)
- You're writing a method that relates to the class, not just a single object

---

🏗️ 4. Real-Life Example: Alternate Constructor

Class methods can act as **alternate constructors** — ways to create objects differently.

Example 👇

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

    @classmethod
    def from_string(cls, data_str):
        name, marks = data_str.split("-")
        return cls(name, int(marks))  # creating object from string

# Creating object using normal constructor
s1 = Student("Alice", 85)

# Creating object using class method
s2 = Student.from_string("Bob-90")

print(s1.name, s1.marks)
print(s2.name, s2.marks)
```

📜 Output:

```
Alice 85
Bob 90
```

✅ The from_string() method gives us a **custom way to create objects** — super useful when parsing data from files, APIs, or databases.

🧠 5. Difference Between Instance, Class, and Static Methods

Type	First Argument	Access To	When To Use
Instance Method	self	Object attributes	When method works on individual object
Class Method	cls	Class variables	When you want to access/modify class data
Static Method	None	Neither	When the method is independent of class/object

---

Example of all three:

```
class Demo:
    company = "PythonSoft"

    def __init__(self, name):
        self.name = name

    def show(self):  # Instance method
        print(f"Employee: {self.name}, Company: {self.company}")

    @classmethod
    def change_company(cls, new_name):  # Class method
        cls.company = new_name

    @staticmethod
    def info():  # Static method
        print("This is a PythonSoft Employee class")
```

```
# Using all three
d1 = Demo("John")
d1.show()

Demo.change_company("NextGenSoft")
d1.show()

Demo.info()
```

📜 Output:

```
Employee: John, Company: PythonSoft
Employee: John, Company: NextGenSoft
This is a PythonSoft Employee class
```

💡 6. Key Takeaways

- Use @classmethod when you need to **work with class variables** or create **alternative constructors**.
- Use cls (not self).
- It can be called on both **class** and **object**.

✅ Example of both ways:

```
class Test:
    @classmethod
    def hello(cls):
        print("Hello from class method!")

Test.hello()   # called using class
```

```
obj = Test()
obj.hello()    # called using object (also works)
```

🧠 Even though you can call it via an object, it always refers to the class (cls).

🧪 7. Practice Exercise

Try this yourself 👇

```
# Exercise: Class Method Practice
# 1. Create a class 'Book' with:
#    - class variable: total_books = 0
#    - instance variables: title, author
# 2. Every time a new book is created, increment total_books.
# 3. Add a class method 'book_count()' that prints total_books.
# 4. Add an alternate constructor 'from_string("Title-Author")'.
# 5. Create a few books using both constructors and test everything.
```

📜 8. Summary Table

Keyword	First Arg	Purpose	Access	Example
self	Object	Works on instance data	Instance variables	def show(self):
cls	Class	Works on class data	Class variables	@classmethod
(none)	None	Independent utility	None	@staticmethod